LESSON — SORTING ALGORITHMS

What This Lesson Is About

This lesson covers sorting, which means putting data into a chosen order, and the algorithms that do it. Since every algorithm does the same job, they differ only in speed (time complexity), extra memory (space complexity), and stability (whether equal items keep their original order). The slides teach three simple sorts (Bubble, Selection, Insertion) and four advanced ones (Shell, Counting, Bucket, Merge), and the exam mostly asks you to tell them apart.

Definition

Sorting is the process of arranging a collection of data according to a particular order. The most common orderings are **ascending** (smallest to largest) and **descending** (largest to smallest).

Example: *Unsorted: [45, 12, 78, 23, 9, 34] → Ascending: [9, 12, 23, 34, 45, 78] → Descending: [78, 45, 34, 23, 12, 9].*

> ***Situation:*** *Always read the direction of the order before you start tracing. Every trace in this reviewer sorts in **ascending** order, but an exam item can ask for descending the algorithm is the same, only the comparison flips (> becomes <).*

Why Do We Need Sorting?

- **Faster searching** — a sorted collection allows more efficient searching. Binary search, for example, *requires* sorted data. Unsorted [45, 12, 78, 23, 9, 34] must be sorted to [9, 12, 23, 34, 45, 78] before binary search can be used.
- **Easier analysis** — sorting makes it easy to identify minimum, maximum and median values, rankings, duplicates and ranges. Grades 78, 92, 65, 88, 95, 73 sorted into 65, 73, 78, 88, 92, 95 immediately show the ranking and distribution.
- **Data presentation** — student lists, product catalogs, employee records, rankings, search results and transaction histories are all displayed in sorted order.

> ***Situation:*** *A typical short-answer item is "give two reasons sorting is useful." The safest pair to write: (1) it enables faster searching such as binary search, and (2) it makes min/max/median/ranking/duplicates easy to find.*

Terminologies

Pass

One complete sweep through the (unsorted part of the) array. Bubble Sort and Selection Sort are described in passes; Insertion Sort handles one new element ("key") per pass.

Stable Sort

A sort is **stable** if elements with equal key values keep their original relative order in the output. If two 5s appear in the input, the one that came first is still first after sorting.

Example: *Records (5a, 3, 5b) sorted by number: a stable sort gives (3, 5a, 5b). An unstable sort might give (3, 5b, 5a).*

In-Place Sort

A sort is **in-place** if it needs only a constant amount of extra memory, O(1), beyond the input array itself.

Comparison-Based vs. Non-Comparison-Based

A **comparison-based** sort decides order by comparing elements with each other (Bubble, Selection, Insertion, Shell, Merge). A **non-comparison-based** sort uses the *values themselves* — counting occurrences or distributing into groups — and never compares two elements directly (Counting, Bucket).

Best, Average and Worst Case

The **best case** is the input that makes the algorithm fastest (often already-sorted data), the **worst case** is the input that makes it slowest (often reverse-sorted data), and the **average case** is the typical random input. **Space** is the extra memory used. All of these are written in Big-O notation, e.g. O(n), O(n²), O(n log n).

> ***Situation:*** *"Stable" and "in-place" are two **independent** properties. Selection Sort is in-place but not stable; Merge Sort is stable but not in-place. Do not assume one implies the other.*

Basic Classification of Sorting Algorithms

Group	Algorithms	General Idea
Simple / Basic	Bubble Sort, Selection Sort, Insertion Sort	Easier to understand and implement, but O(n²) worst-case time.

Group	Algorithms	General Idea
Advanced	Merge Sort, Quick Sort, Heap Sort, Shell Sort, Counting Sort, Radix Sort, Bucket Sort	More sophisticated strategies; can perform better on appropriate types of input.

> ***Situation:*** *The slides name Quick, Heap and Radix Sort in the classification but only teach Shell, Counting, Bucket and Merge in detail. For the exam, expect to be able to list all seven advanced names, and to explain and trace the four that were taught.*

Bubble Sort

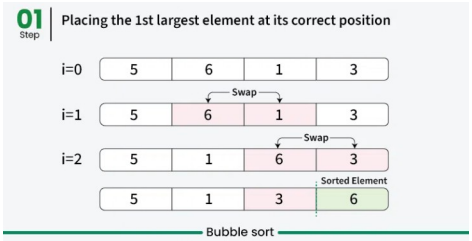
Bubble Sort repeatedly compares **adjacent** elements and swaps them when they are in the wrong order. After each pass the largest remaining element "bubbles up" to its correct position at the end of the array.

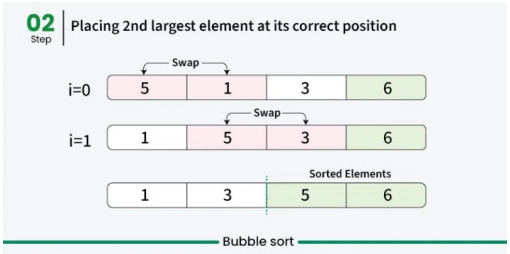
Steps (ascending):

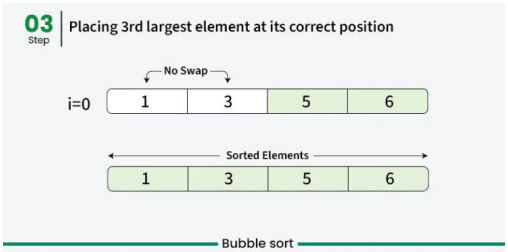
1. Start at the first element and compare it with the next one.
2. If the left element is bigger than the right one, swap them.
3. Move one position right and repeat until the end of the unsorted part. The largest element is now in its final place.
4. Repeat the whole sweep for the remaining unsorted part. Stop when a pass makes no swaps (or after n − 1 passes).

```
for i = 0 to n-2:
    swapped = false
    for j = 0 to n-2-i:
        if arr[j] > arr[j+1]:
            swap(arr[j], arr[j+1]); swapped = true
    if not swapped: break   // O(n) best case
```

The slide diagrams below sort [5, 6, 1, 3]: step 1 places the 1st largest (6) at the end, step 2 places the 2nd largest (5), and step 3 finds no swap needed, so the array is sorted.







Complexity

Case	Big-O	When it happens
Best	O(n) *(with early-exit check)*	Array is already sorted — one pass, no swaps.
Average	O(n²)	Random order.
Worst	O(n²)	Array is in reverse order.
Space	O(1)	Swaps happen in place.

> ***Situation:*** *On the slide, the Bubble Sort best case is marked with an asterisk. The O(n) best case is only true when the algorithm is written with the **swapped flag** (stop if a pass makes no swaps). A version without the flag always does O(n²) comparisons.*

Advantages

- Easy to understand and implement.
- Needs no additional memory — O(1) space.
- **Stable** — equal elements keep their relative order.

Disadvantages

- O(n²) time makes it very slow for large data sets.

- Almost no real-world use; it is mostly used in academics to teach sorting.

Selection Sort

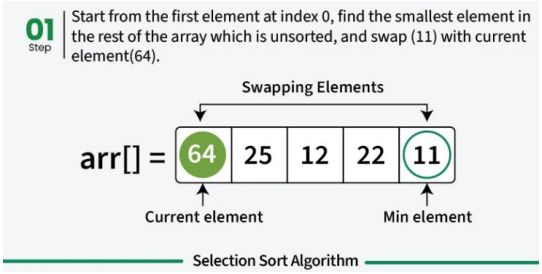
Selection Sort is a comparison-based algorithm that repeatedly **selects the smallest (or largest) element** from the unsorted portion and **swaps it with the first unsorted element**. After pass i, the first i positions hold the i smallest values in their final places.

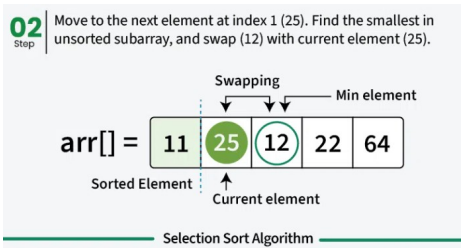
Steps (ascending):

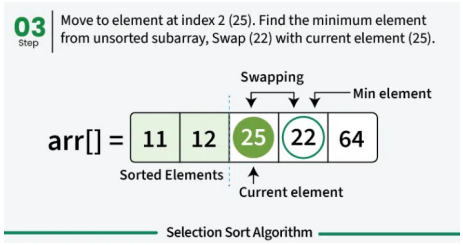
1. Find the smallest element in the whole array and swap it with the first element.
2. Find the smallest among the remaining elements (the second smallest overall) and swap it with the second element.
3. Keep going until every element has been moved to its correct position.

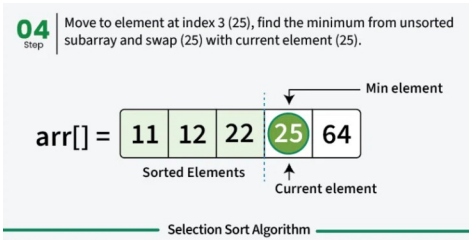
```
for i = 0 to n-2:
    min = i
    for j = i+1 to n-1:
        if arr[j] < arr[min]: min = j
    swap(arr[i], arr[min])
```

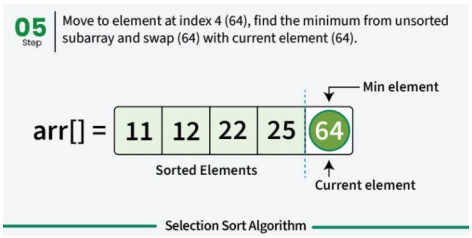
The slide diagrams sort [64, 25, 12, 22, 11]. In the first step the minimum is 11 (index 4), so 11 is swapped with the current element 64; in step 2 the minimum of the rest is 12, which is swapped with 25; and so on until the sorted array [11, 12, 22, 25, 64] appears.

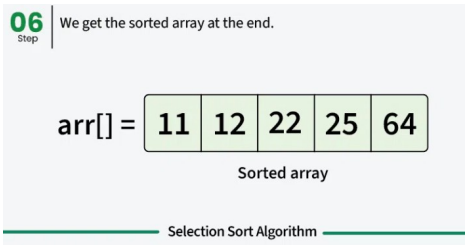












Complexity

Case	Big-O	Why
Best	$O(n^2)$	Still scans the whole unsorted part every pass, even if already sorted.
Average	$O(n^2)$	Always about $n^2/2$ comparisons.
Worst	$O(n^2)$	Same number of comparisons.
Space	O(1)	In place.

> ***Situation:*** *Selection Sort is the **only basic sort whose best case is also $O(n^2)$** — its running time does not depend on the input order. It does, however, make at most $n - 1$ swaps, the fewest of the three basic sorts. That is why "which sort requires the fewest swaps / memory writes?" has Selection Sort as its answer.*

Advantages

- Easy to understand and implement — ideal for teaching basic sorting.
- Constant O(1) extra memory.
- Requires fewer swaps (memory writes) than many other standard algorithms.

Disadvantages

- O(n²) time — slower than Quick Sort or Merge Sort.
- **Not stable** — the long-distance swap can jump an element over an equal one.

Example: *[4a, 4b, 2]: pass 1 swaps the minimum 2 with the first element 4a, giving [2, 4b, 4a]. The two 4s have changed order, so Selection Sort is not stable.*

Insertion Sort

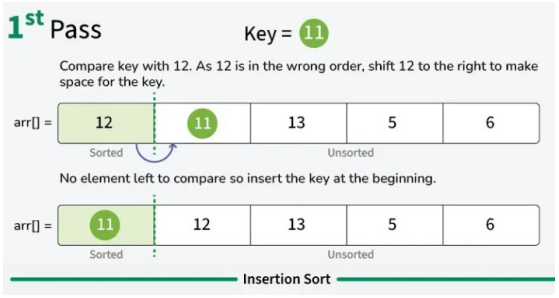
Insertion Sort works by iteratively **inserting each element of the unsorted part into its correct position in the sorted part** — exactly like sorting playing cards in your hand. The element being inserted is called the **key**.

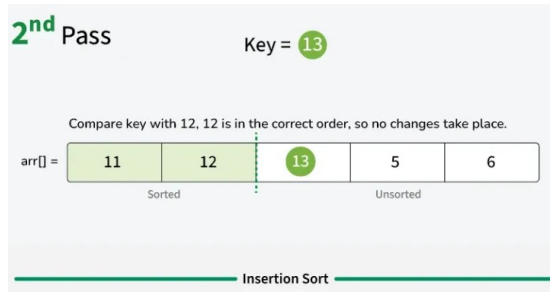
Steps (ascending):

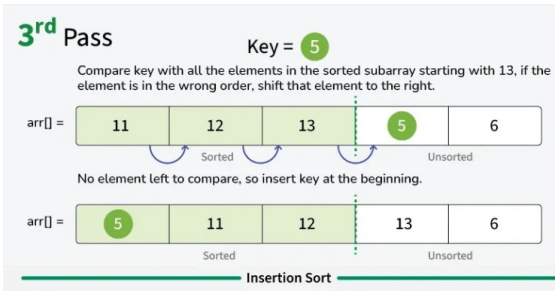
1. Start with the second element; the first element is assumed to be sorted.
2. Compare the key with the elements to its left; shift every larger element one place to the right.
3. Insert the key into the gap that opens up. Move to the next element.
4. Repeat until the entire array is sorted.

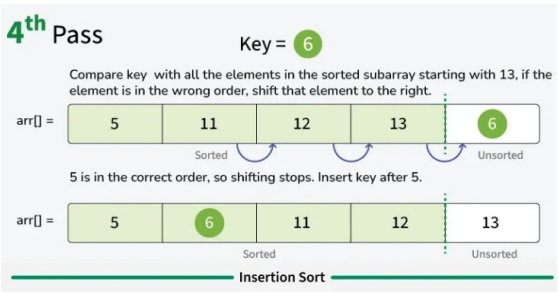
```
for i = 1 to n-1:
    key = arr[i]; j = i-1
    while j >= 0 and arr[j] > key:
        arr[j+1] = arr[j]; j = j-1
    arr[j+1] = key
```

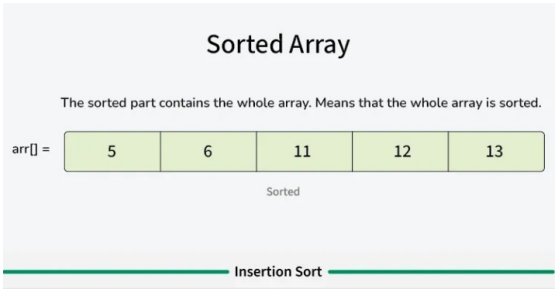
The slide diagrams sort [12, 11, 13, 5, 6]. Pass 1 (key 11): 12 shifts right and 11 goes first. Pass 2 (key 13): 13 is already in the right place, so nothing changes. Pass 3 (key 5): 13, 12 and 11 all shift right and 5 goes to the front. Pass 4 (key 6): 13, 12 and 11 shift right and 6 is inserted after 5.











Complexity

Case	Big-O	When it happens
Best	O(n)	Already sorted — each key needs just one comparison, no shifts.
Average	O(n²)	Random order.
Worst	O(n²)	Reverse order — every key shifts all the way to the front.

Case	Big-O	When it happens
Space	O(1)	In place; also **stable**.

> ***Situation:*** *Insertion Sort is the best of the three basic sorts on **nearly sorted** data, because the inner while-loop barely runs. This is also why Shell Sort (below) is built on top of it.*

Comparison of the Basic Algorithms

All three have O(n²) average and worst-case performance, which means their execution time grows rapidly as the input gets larger.

Algorithm	Best	Avg / Worst	Stable?
Bubble Sort	O(n)	O(n²)	Yes
Selection Sort	O(n²)	O(n²)	No
Insertion Sort	O(n)	O(n²)	Yes

> ***Situation:*** *Memorize the odd one out: Selection Sort is the one with an O(n²) **best** case and the one that is **not stable**. Bubble and Insertion are both O(n) best-case and stable.*

Shell Sort

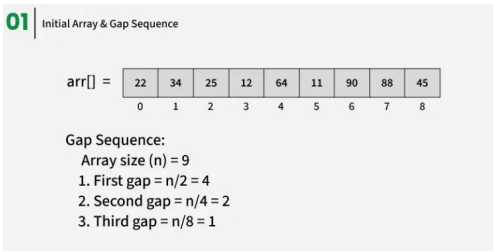
Shell Sort (Shell's method) is an **in-place comparison sort** and an **optimization of Insertion Sort**. It was considered the first algorithm to break the O(n²) time barrier for sorting. It works by comparing elements that are **far apart first**, then gradually **reducing the gap**, which lets elements travel long distances across the array in a single move.

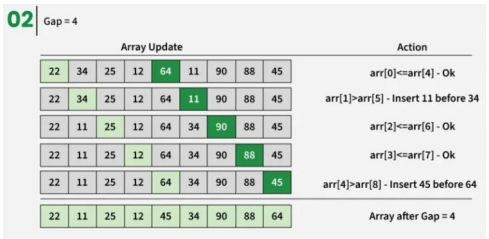
Steps:

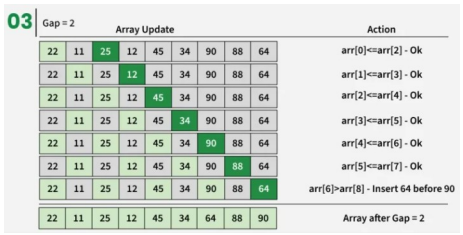
1. Choose a gap sequence — commonly n/2, n/4, …, 1.
2. For the current gap, sort the elements that are *gap* positions apart using Insertion Sort.
3. Reduce the gap and repeat. The last pass is always gap = 1 (plain Insertion Sort on a nearly sorted array).

```
gap = n / 2
while gap >= 1:
   for i = gap to n-1:
      temp = arr[i]; j = i
      while j >= gap and arr[j-gap] > temp:
         arr[j] = arr[j-gap]; j = j-gap
      arr[j] = temp
   gap = gap / 2
```

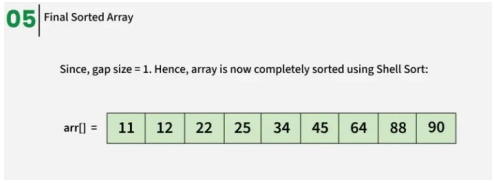
The slide diagrams sort [22, 34, 25, 12, 64, 11, 90, 88, 45] (n = 9). Gap sequence: n/2 = 4, then n/4 = 2, then n/8 = 1 (integer division). The array after each gap is shown, ending in [11, 12, 22, 25, 34, 45, 64, 88, 90].











Complexity

Case	Big-O	Notes
Best	O(n log n)	Array already (nearly) sorted; depends on the gap sequence.
Average	Between O(n^1.25) and O(n^1.5)	Depends on the chosen gap sequence.
Worst	O(n²)	With simple gap sequences such as n/2, n/4, …, 1.
Space	O(1)	In place.

Advantages

- In-place — only O(1) extra space.
- Faster than Insertion Sort, especially on larger arrays.
- Easy to implement — a straightforward improvement over Insertion Sort.

Disadvantages

- **Not stable.**
- Performance depends on the gap sequence; bad choices can degrade it to O(n²).
- Merge Sort and Quick Sort usually outperform it on large data sets.

> ***Situation:*** *Shell Sort =* ***Insertion Sort with gaps****. If a question asks "which advanced sort is an optimization of Insertion Sort?" the answer is Shell Sort. And remember: the final pass always uses gap = 1.*

Counting Sort

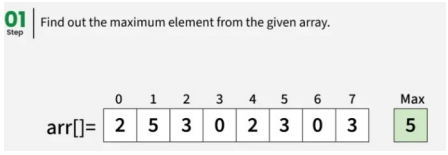
Counting Sort is a **non-comparison-based** sorting algorithm. It is particularly efficient when the **range of input values is small** compared to the number of elements. The idea is to **count the frequency of each distinct value** and use those counts to place every element directly into its correct sorted position.

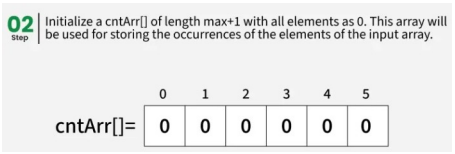
Steps:

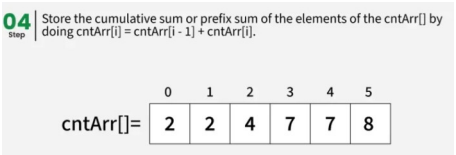
1. Find the maximum element of the input array.
2. Create a count array cntArr[0..max] filled with 0, then count how many times each value occurs.
3. Turn the count array into a **cumulative (prefix) sum**: cntArr[i] = cntArr[i−1] + cntArr[i]. Now cntArr[v] tells you the position just past the last v in the output.
4. Traverse the input array **in reverse**. For each value, put it at ans[cntArr[value] − 1], then decrement cntArr[value]. Going in reverse keeps equal elements in order — that is what makes the sort **stable**.

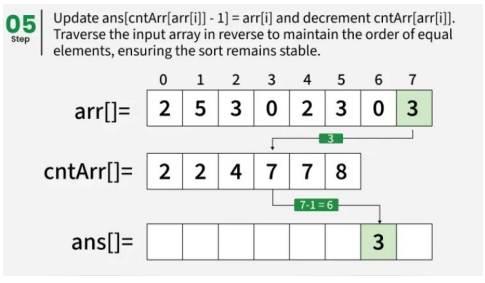
```
max = largest value in arr
cnt[0..max] = 0
for each x in arr: cnt[x]++
for i = 1 to max:          // prefix sums
   cnt[i] = cnt[i] + cnt[i-1]
for i = n-1 down to 0:     // reverse = stable
   ans[cnt[arr[i]] - 1] = arr[i]
   cnt[arr[i]]--
```

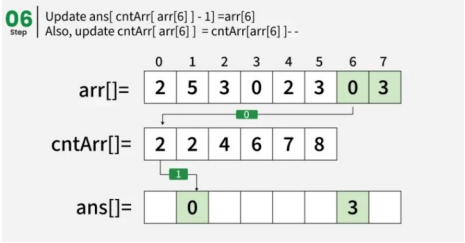
The slide diagrams sort arr = [2, 5, 3, 0, 2, 3, 0, 3]. Max is 5, so cntArr has indexes 0 to 5. Counting the occurrences gives cntArr = [2, 0, 2, 3, 0, 1] (the slides skip this as step 03), and taking prefix sums gives [2, 2, 4, 7, 7, 8] (step 04). Steps 5 to 12 then place the elements from the right end of arr into ans[], giving [0, 0, 2, 2, 3, 3, 3, 5].

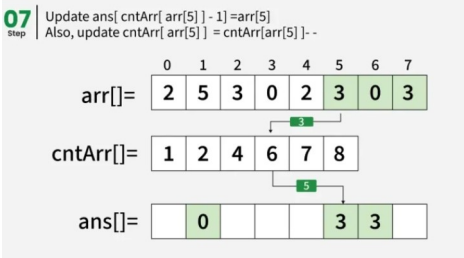


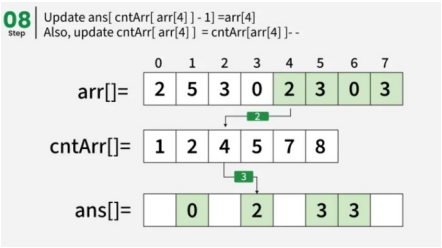


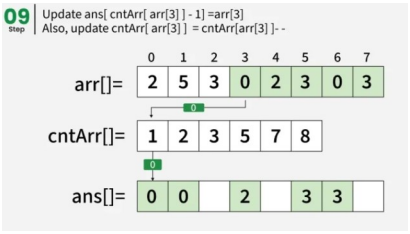


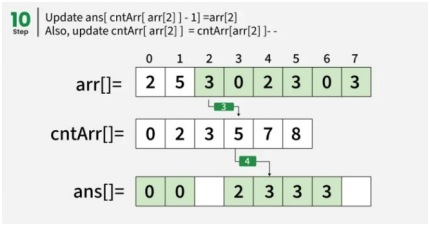


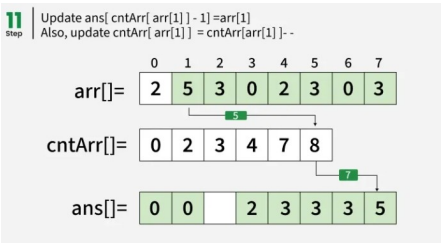


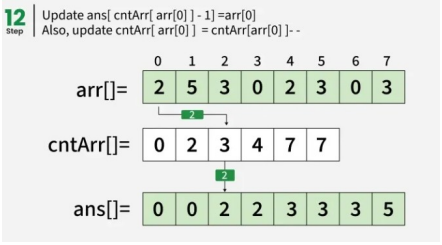












Complexity

Measure	Big-O	Where N = size of input array, M = size of count array
Time (all cases)	O(N + M)	Count the input (N), build prefix sums (M), place the output (N).
Auxiliary space	O(N + M)	Output array (N) plus count array (M).

Advantages

- Faster than comparison sorts such as Merge Sort and Quick Sort when the range of input is of the order of the number of inputs.
- **Stable** algorithm.

Disadvantages

- Does not work on decimal values.
- Inefficient if the range of values is very large.
- **Not in-place** — uses extra space for the count and output arrays.

> ***Situation:*** *A classic trap: Counting Sort can beat the O(n log n) limit of comparison sorts only because it never compares elements. But it pays for that with memory proportional to the value range M, so sorting [3, 1000000, 7] with Counting Sort is a bad idea.*

Bucket Sort

Bucket Sort divides the elements into groups called **buckets**, formed by uniformly distributing the elements. Each bucket is then sorted with another sorting algorithm (the slides use Insertion Sort), and finally the buckets are gathered together in order. It works well when the input is **uniformly distributed across a range**, and it is **stable** because Insertion Sort (which is stable) sorts the individual buckets.

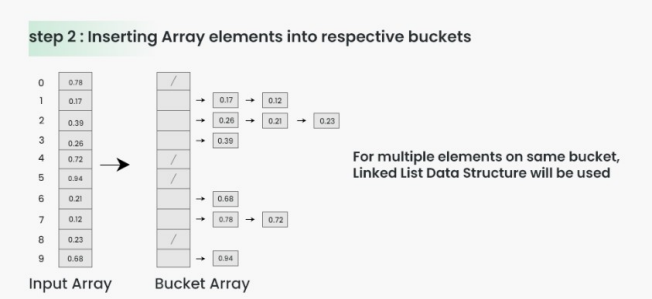
Steps:

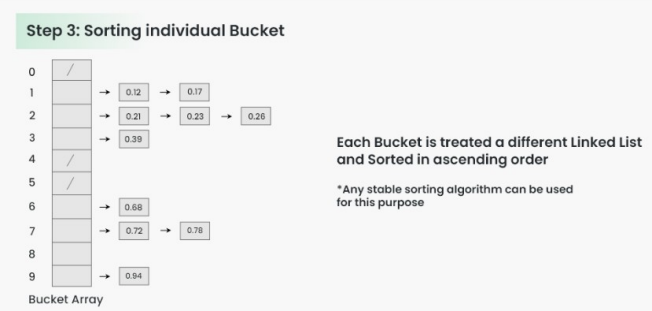
1. Create n empty buckets (or lists).
2. For every array element arr[i], insert it into bucket[n × arr[i]] (for values in the range 0 to just under 1).
3. Sort each individual bucket using Insertion Sort.
4. Concatenate all the sorted buckets in order.

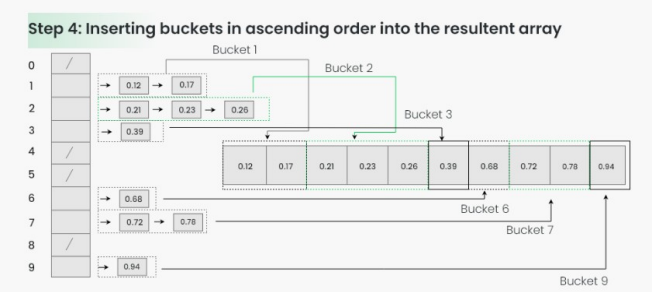
```
create n empty buckets
for each x in arr:
    insert x into bucket[ floor(n * x) ]
for each bucket: insertion_sort(bucket)
output = bucket[0] + bucket[1] + ... + bucket[n-1]
```

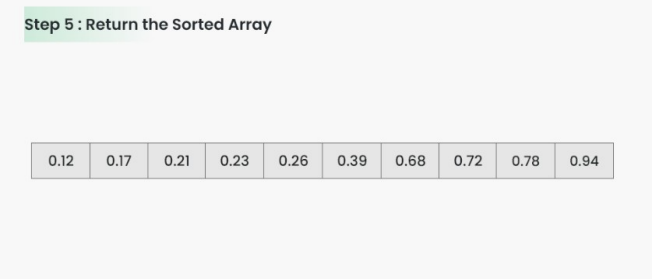
The slide diagrams sort 10 numbers: [0.78, 0.17, 0.39, 0.26, 0.72, 0.94, 0.21, 0.12, 0.23, 0.68] using 10 buckets. 0.78 goes to bucket ⌊10 × 0.78⌋ = 7, 0.17 to bucket 1, 0.39 to bucket 3, and so on. Buckets that receive several values (like bucket 2: 0.26, 0.21, 0.23) are stored as linked lists, then sorted, then joined to give [0.12, 0.17, 0.21, 0.23, 0.26, 0.39, 0.68, 0.72, 0.78, 0.94].











Complexity

Case	Big-O	Notes
Best	O(n + k)	Elements spread evenly across k buckets.
Average	O(n + k)	Uniformly distributed input.
Worst	O(n²)	Everything lands in one bucket, so the inner sort does all the work.
Space	O(n + k)	Extra memory for the buckets.

Advantages

- Fast when data is uniformly distributed over a known range.
- Conceptually simple: distribute, sort each bucket, combine.
- Can be stable if the inner sorting algorithm is stable and ordering is preserved.
- Buckets can be processed independently, so some implementations suit parallel processing.

Disadvantages

- Performance depends heavily on the distribution of the input.
- Requires additional memory for the buckets.
- Choosing an appropriate number of buckets is difficult.
- Poor bucket selection gives unbalanced buckets and reduces performance.

> ***Situation:*** *Bucket Sort and Counting Sort are both non-comparison sorts, but they target different inputs:* ***Counting Sort*** *wants small whole numbers with a small range;* ***Bucket Sort*** *wants numbers spread uniformly over a range (often decimals between 0 and 1). If everything crowds into one bucket, Bucket Sort collapses to O(n²).*

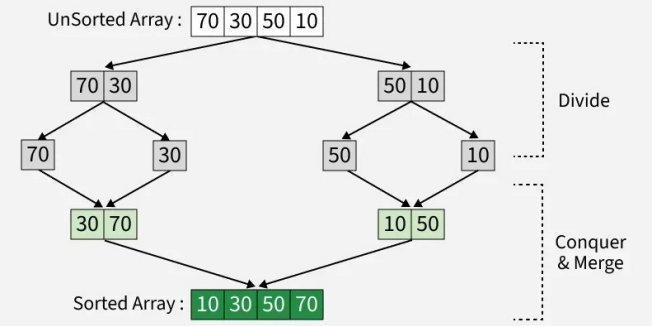
Merge Sort

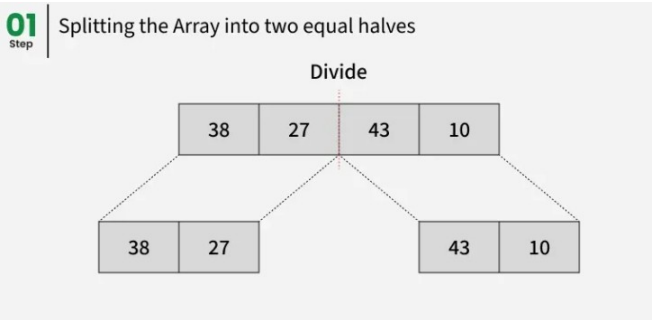
Merge Sort recursively **divides** the array into two halves, **sorts** each half, and then **merges** the two sorted halves back together. It is a classic *divide-and-conquer* algorithm.

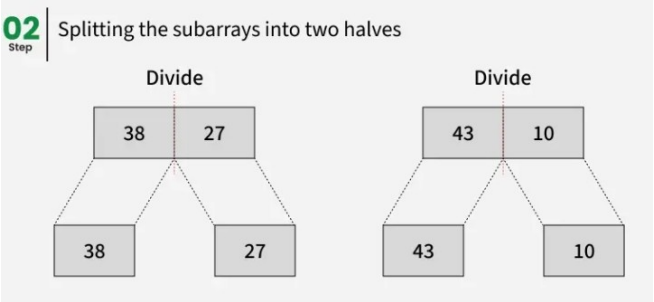
Phase	What happens
Divide	Split the list recursively into two halves until it can no longer be divided (single elements).
Conquer	Each subarray is sorted individually using Merge Sort. A one-element array is already sorted.
Merge	Merge the sorted subarrays back together in sorted order, continuing until all elements are merged.

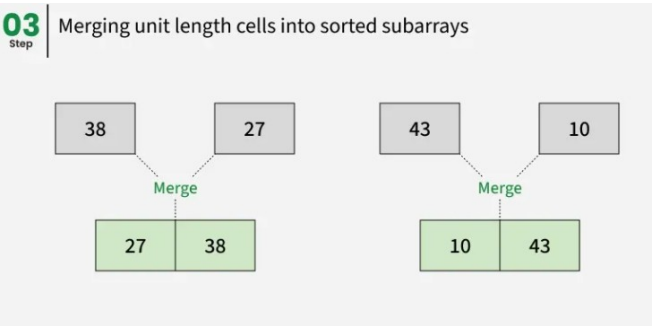
```
mergeSort(arr, l, r):
    if l < r:
        m = (l + r) / 2
        mergeSort(arr, l, m)      // left half
        mergeSort(arr, m+1, r)    // right half
        merge(arr, l, m, r)       // combine
```

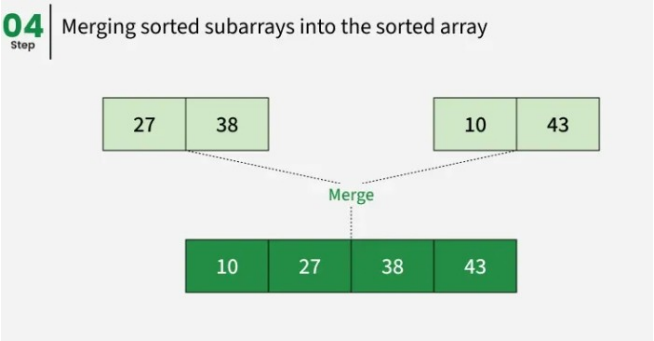
The first diagram splits [70, 30, 50, 10] down to single elements, merges up to [30, 70] and [10, 50], and finally to [10, 30, 50, 70]. The step diagrams show [38, 27, 43, 10]: split into [38, 27] and [43, 10], split again into single cells, merge into [27, 38] and [10, 43], and merge into [10, 27, 38, 43].











Complexity

Case	Big-O	Notes
Best	O(n log n)	log n levels of splitting, each level merges n elements.
Average	O(n log n)	Same.
Worst	O(n log n)	Guaranteed — does not depend on input order.
Space	O(n)	Needs a temporary array to merge.

Advantages

- **Stable** — keeps equal elements in order.
- Guaranteed O(n log n) worst case, even on large data sets.
- Simple to implement — the divide-and-conquer approach is straightforward.
- Naturally parallel — subarrays can be merged independently.

Disadvantages

- **Not in-place** — needs additional memory to store the sorted data, a concern when memory is limited.
- Generally slower than Quick Sort, because Quick Sort works in place and is more cache-friendly.

> ***Situation:*** *Merge Sort is the "safe" choice: it never degrades below O(n log n) and it is stable. The price is O(n) extra memory. If a question says "guaranteed performance **and** stability," the answer is Merge Sort.*

Master Comparison — All Seven Algorithms

Algorithm	Best	Average	Worst
Bubble	O(n)	O(n²)	O(n²)
Selection	O(n²)	O(n²)	O(n²)
Insertion	O(n)	O(n²)	O(n²)
Shell	O(n log n)	O(n^1.25)–O(n^1.5)	O(n²)
Counting	O(N+M)	O(N+M)	O(N+M)
Bucket	O(n+k)	O(n+k)	O(n²)
Merge	O(n log n)	O(n log n)	O(n log n)

Algorithm	Space	Stable?	In-place?	Type
Bubble	O(1)	Yes	Yes	Comparison
Selection	O(1)	No	Yes	Comparison
Insertion	O(1)	Yes	Yes	Comparison
Shell	O(1)	No	Yes	Comparison
Counting	O(N+M)	Yes	No	Non-comparison
Bucket	O(n+k)	Yes*	No	Non-comparison
Merge	O(n)	Yes	No	Comparison

Note: the asterisk on Bucket Sort's "Yes" means it is stable only when the sort used inside each bucket is stable (the slides use Insertion Sort).

Which algorithm fits which situation?

If the question says…	Think of…
Easy to teach / simplest idea	Bubble Sort
Fewest swaps (memory writes)	Selection Sort
Data is already nearly sorted	Insertion Sort (or Bubble with the early-exit flag)
Improvement of Insertion Sort using gaps	Shell Sort
Small range of whole numbers, no comparisons	Counting Sort
Uniformly distributed values (e.g. decimals 0 to 1)	Bucket Sort
Guaranteed O(n log n) and stable	Merge Sort
Divide and conquer	Merge Sort

Quick Self-Check — Sorting Algorithms

- What does it mean for a sort to be stable? Which of the seven algorithms are not stable?
- Why is the best case of Bubble Sort O(n) but the best case of Selection Sort still O(n²)?
- What is the key in Insertion Sort, and what happens to the elements larger than it?
- How does Shell Sort improve on Insertion Sort, and what gap does the last pass always use?
- Why does Counting Sort traverse the input array in reverse when filling the output?
- When does Bucket Sort degrade to O(n²)?
- What are the divide, conquer and merge phases of Merge Sort, and why does it need O(n) extra space?

PART II — FULLY WORKED TRACES

Below, one fresh example is traced for every algorithm, step by step, so you can reproduce the method on a new set of numbers during the exam. Every trace sorts in ascending order. Each row shows the array *after* that pass.

Trace 1 — Bubble Sort on [29, 10, 14, 37, 13]

Compare neighbours left to right and swap when the left one is bigger. After pass i the i largest values sit at the end.

Pass	Array after pass	Swaps	What happened
1	[10, 14, 29, 13, 37]	3	Largest unsorted value moved to the end of the unsorted part.
2	[10, 14, 13, 29, 37]	1	Largest unsorted value moved to the end of the unsorted part.
3	[10, 13, 14, 29, 37]	1	Largest unsorted value moved to the end of the unsorted part.
4	[10, 13, 14, 29, 37]	0	No swaps — the array is sorted, so the early-exit check stops the algorithm.

Result: [10, 13, 14, 29, 37]. Total swaps = 5. The last pass makes no swaps, which is how the algorithm knows to stop.

> ***Situation:*** *Pass 1 in detail: compare 29 and 10 → swap; 29 and 14 → swap; 29 and 37 → no swap; 37 and 13 → swap. The biggest value, 37, has bubbled to the end after 3 swaps.*

Trace 2 — Selection Sort on [64, 25, 12, 22, 11]

In each pass find the minimum of the unsorted part and swap it with the first unsorted element. This matches the slide example.

Pass	Minimum found	Swapped with	Array after pass
1	11	64	[11, 25, 12, 22, 64]
2	12	25	[11, 12, 25, 22, 64]
3	22	25	[11, 12, 22, 25, 64]
4	25	25	[11, 12, 22, 25, 64]

Result: [11, 12, 22, 25, 64]. Only 4 swaps were needed (n − 1 = 4).

Trace 3 — Insertion Sort on [12, 11, 13, 5, 6]

Treat the first element as sorted; take each next element as the key and shift larger sorted elements right. This matches the slide example.

Pass	Key	Elements shifted right	Array after pass
1	11	1	[11, 12, 13, 5, 6]
2	13	0 (already in place)	[11, 12, 13, 5, 6]
3	5	3	[5, 11, 12, 13, 6]
4	6	3	[5, 6, 11, 12, 13]

Result: [5, 6, 11, 12, 13].

Trace 4 — Shell Sort on [35, 33, 42, 10, 14, 19, 27, 44]

n = 8, so the gap sequence is n/2 = 4, then 2, then 1.

Gap	Pairs compared / sublists sorted	Array after this gap
4	(35,14), (33,19), (42,27), (10,44) — each pair is put in order	[14, 19, 27, 10, 35, 33, 42, 44]
2	Sublist at even positions [14, 27, 35, 42] is already ordered; sublist at odd positions [19, 10, 33, 44] becomes [10, 19, 33, 44]	[14, 10, 27, 19, 35, 33, 42, 44]
1	Ordinary Insertion Sort on a nearly sorted array — only a few small shifts	[10, 14, 19, 27, 33, 35, 42, 44]

Result: [10, 14, 19, 27, 33, 35, 42, 44]. Notice how 14 jumped from index 4 to index 0 (and 19 from index 5 to index 1) in a single gap-4 move — that long-distance movement is what plain Insertion Sort cannot do.

Trace 5 — Counting Sort on [4, 2, 2, 8, 3, 3, 1]

- **Step 1 — Max:** the largest element is 8, so the count array has indexes 0 to 8.
- **Step 2 — Count occurrences:** cntArr = [0, 1, 2, 2, 1, 0, 0, 0, 1] (index = value, entry = how many times it appears).
- **Step 3 — Prefix sums:** cntArr = [0, 1, 3, 5, 6, 6, 6, 6, 7] (each entry = previous entry + itself).
- **Step 4 — Place from the right end of the input:** for each value x, ans[cntArr[x] − 1] = x, then decrement cntArr[x].

Input read (right → left)	Value	Goes to index	ans[] so far
arr[6]	1	0	[1, _, _, _, _, _, _]
arr[5]	3	4	[1, _, _, _, 3, _, _]
arr[4]	3	3	[1, _, _, 3, 3, _, _]
arr[3]	8	6	[1, _, _, 3, 3, _, 8]
arr[2]	2	2	[1, _, 2, 3, 3, _, 8]
arr[1]	2	1	[1, 2, 2, 3, 3, _, 8]
arr[0]	4	5	[1, 2, 2, 3, 3, 4, 8]

Result: [1, 2, 2, 3, 3, 4, 8]. The two 2s and the two 3s kept their relative order because the input was read in reverse, so the sort is **stable**.

Trace 6 — Bucket Sort on [0.42, 0.32, 0.23, 0.52, 0.25, 0.47, 0.51]

n = 7 values, so create 7 buckets (index 0 to 6) and send each value x to bucket ⌊7 × x⌋.

Value x	7 × x	Bucket index
0.42	2.94	2
0.32	2.24	2
0.23	1.61	1
0.52	3.64	3
0.25	1.75	1
0.47	3.29	3
0.51	3.57	3

Bucket	Contents after distribution	Contents after sorting the bucket
0	(empty)	(empty)
1	[0.23, 0.25]	[0.23, 0.25]
2	[0.42, 0.32]	[0.32, 0.42]
3	[0.52, 0.47, 0.51]	[0.47, 0.51, 0.52]
4	(empty)	(empty)
5	(empty)	(empty)
6	(empty)	(empty)

Result (concatenate buckets 0 → 6): [0.23, 0.25, 0.32, 0.42, 0.47, 0.51, 0.52]. Bucket 3 got three values, so most of the work happened there — if all 7 values had landed in one bucket, the inner Insertion Sort would do all the work and the cost would approach $O(n^2)$.

Trace 7 — Merge Sort on [38, 27, 43, 3, 9, 10]

Every split divides a list into a left and a right half (the left half gets the smaller share when the length is odd). Every merge repeatedly takes the smaller front element of the two sorted halves.

Divide phase and merge phase, in the order they happen:

- **Split** [38, 27, 43, 3, 9, 10] → [38, 27, 43] | [3, 9, 10]
- **Split** [38, 27, 43] → [38] | [27, 43]
- **Split** [27, 43] → [27] | [43]
- **Merge** [27] + [43] → [27, 43]
- **Merge** [38] + [27, 43] → [27, 38, 43]
- **Split** [3, 9, 10] → [3] | [9, 10]
- **Split** [9, 10] → [9] | [10]
- **Merge** [9] + [10] → [9, 10]
- **Merge** [3] + [9, 10] → [3, 9, 10]
- **Merge** [27, 38, 43] + [3, 9, 10] → [3, 9, 10, 27, 38, 43]

Result: [3, 9, 10, 27, 38, 43].

> ***Situation:*** *The merge step is where the sorting really happens. Take the final merge above, [27, 38, 43] + [3, 9, 10]: compare 27 with 3 → take 3; 27 with 9 → take 9; 27 with 10 → take 10; the right side is now empty, so copy 27, 38, 43. Result: [3, 9, 10, 27, 38, 43].*

PART III — PRACTICE EXAM

Use this as a timed self-test after you have reviewed Parts I and II. The answer key is at the very end — don't peek early.

I. Matching Type

#	Column Y (Term)
1	Sorting
2	Stable Sort
3	In-Place Sort
4	Bubble Sort
5	Selection Sort
6	Insertion Sort
7	Shell Sort
8	Counting Sort
9	Bucket Sort
10	Merge Sort

Match each term in Column Y to its description in Column Z. Write the letter on the blank. Some letters in Column Z will not be used.

Column Z

a. Recursively divides the array into halves, sorts each half, then merges the halves back together.

b. The process of arranging a collection of data according to a particular order.

c. A non-comparison sort that counts how often each value occurs and uses prefix sums to place elements.

d. A sort that keeps elements with equal keys in their original relative order.

e. Repeatedly compares adjacent elements and swaps them when they are in the wrong order.

f. An optimization of Insertion Sort that compares far-apart elements first, using a shrinking gap.

g. A sort that needs only a constant amount of extra memory, O(1), besides the input.

h. Finds a value in a sorted list by repeatedly halving the search range.

i. Takes each element of the unsorted part and inserts it into its correct place in the sorted part.

j. Repeatedly picks the smallest element of the unsorted part and swaps it with the first unsorted element.

k. Spreads elements uniformly into groups, sorts each group, then concatenates the groups.

II. Modified True or False

Write "TRUE" if the statement is correct. Otherwise, underline the false word/phrase and write the correction on the blank. (2 points each)

1. Selection Sort is a **stable** sorting algorithm.
2. The best-case time complexity of Insertion Sort is **O(n²)**.
3. Merge Sort has a worst-case time complexity of **O(n log n)**.
4. Counting Sort is a **comparison-based** sorting algorithm.
5. Merge Sort is an **in-place** algorithm.
6. Shell Sort is an optimization of **Selection Sort**.
7. Bucket Sort works best when the input is **uniformly distributed** across a range.
8. The average-case time complexity of Bubble Sort is **O(n log n)**.
9. Counting Sort is most efficient when the range of values is **very large** compared with the number of elements.
10. Bubble Sort, Selection Sort and Insertion Sort all have a worst-case time of **O(n²)**.

III. Problem Solving

Show your full solution, not just the final answer.

1. Sort [6, 3, 9, 2, 5] in ascending order using **Bubble Sort**. Show the array after every pass, and state the number of passes made (including the final pass that detects no swaps) and the total number of swaps. (12 points)
2. Sort [20, 12, 10, 15, 2] in ascending order using **Selection Sort**. Show the minimum found and the array after every pass. (10 points)
3. Sort [8, 3, 7, 1, 5] in ascending order using **Insertion Sort**. For every pass, show the key and the array afterwards. (10 points)
4. Sort [3, 1, 4, 1, 3, 0, 4] using **Counting Sort**. Show the count array, the cumulative count array, and the final output. (15 points)
5. Complete the table below for all seven algorithms: best, average and worst time, extra

space, and whether the algorithm is stable. (14 points)

Algorithm	Best	Average	Worst
Bubble			
Selection			
Insertion			
Shell			
Counting			
Bucket			
Merge			

Algorithm	Space	Stable?
Bubble		
Selection		
Insertion		
Shell		
Counting		
Bucket		
Merge		

6. Show the divide phase and the merge phase of **Merge Sort** on [14, 7, 3, 12, 9, 11]. Then state its time complexity and extra space. (14 points)
7. A program must sort 50,000 decimal values that are spread evenly between 0 and 1. Which of the seven algorithms is the best fit, and why? Name one weakness of that algorithm. (5 points)

Answer Key

I. Matching

1-b, 2-d, 3-g, 4-e, 5-j, 6-i, 7-f, 8-c, 9-k, 10-a

II. Modified True/False

1. unstable (not stable)
2. O(n) — when the array is already sorted
3. TRUE
4. non-comparison-based
5. not in-place — it needs O(n) extra space
6. Insertion Sort
7. TRUE
8. $O(n^2)$
9. small
10. TRUE

III. Problem Solving — grade against your reviewer

1. Bubble Sort on [6, 3, 9, 2, 5]:

Pass	Array after pass	Swaps
1	[3, 6, 2, 5, 9]	3
2	[3, 2, 5, 6, 9]	2
3	[2, 3, 5, 6, 9]	1
4	[2, 3, 5, 6, 9]	0

4 passes (the last one finds no swaps and stops the algorithm), 6 swaps in total. Result: [2, 3, 5, 6, 9]. (Part II, Trace 1)

2. Selection Sort on [20, 12, 10, 15, 2]:

Pass	Minimum found	Array after pass
1	2	[2, 12, 10, 15, 20]
2	10	[2, 10, 12, 15, 20]
3	12	[2, 10, 12, 15, 20]
4	15	[2, 10, 12, 15, 20]

Result: [2, 10, 12, 15, 20]. (Part II, Trace 2)

3. Insertion Sort on [8, 3, 7, 1, 5]:

Pass	Key	Array after pass
1	3	[3, 8, 7, 1, 5]
2	7	[3, 7, 8, 1, 5]
3	1	[1, 3, 7, 8, 5]
4	5	[1, 3, 5, 7, 8]

Result: [1, 3, 5, 7, 8]. (Part II, Trace 3)

4. Counting Sort on [3, 1, 4, 1, 3, 0, 4]: max = 4. Count array = [1, 2, 0, 2, 2]. Cumulative count array = [1, 3, 3, 5, 7]. Placing from the right end of the input gives the final output [0, 1, 1, 3, 3, 4, 4]. (Part II, Trace 5)

5. Complexity table:

Algorithm	Best	Average	Worst
Bubble	O(n)	$O(n^2)$	$O(n^2)$
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$
Insertion	O(n)	$O(n^2)$	$O(n^2)$
Shell	O(n log n)	O(n^1.25)–O(n^1.5)	$O(n^2)$
Counting	O(N+M)	O(N+M)	O(N+M)
Bucket	O(n+k)	O(n+k)	$O(n^2)$
Merge	O(n log n)	O(n log n)	O(n log n)

Algorithm	Space	Stable?
Bubble	O(1)	Yes
Selection	O(1)	No
Insertion	O(1)	Yes
Shell	O(1)	No
Counting	O(N+M)	Yes
Bucket	O(n+k)	Yes (stable inner sort)
Merge	O(n)	Yes

6. Merge Sort on [14, 7, 3, 12, 9, 11]:

- Split [14, 7, 3, 12, 9, 11] → [14, 7, 3] | [12, 9, 11]
- Split [14, 7, 3] → [14] | [7, 3]
- Split [7, 3] → [7] | [3]

- Merge [7] + [3] → [3, 7]
- Merge [14] + [3, 7] → [3, 7, 14]
- Split [12, 9, 11] → [12] | [9, 11]
- Split [9, 11] → [9] | [11]
- Merge [9] + [11] → [9, 11]
- Merge [12] + [9, 11] → [9, 11, 12]
- Merge [3, 7, 14] + [9, 11, 12] → [3, 7, 9, 11, 12, 14]

Result: [3, 7, 9, 11, 12, 14]. Time complexity O(n log n) in the best, average and worst case; extra space O(n) because merging needs a temporary array. (Part I, Merge Sort)

7. Bucket Sort is the best fit: the values are uniformly distributed over a known range (0 to 1), so they spread evenly across the buckets and the average time is O(n + k). Weaknesses (any one): performance depends heavily on the distribution — if the values crowd into one bucket the worst case is $O(n^2)$; it needs extra memory for the buckets; and choosing the number of buckets is difficult. (Part I, Bucket Sort)